

FaceLock — Team Handoff & Implementation Guide

Branch: [hardware-ready](#) | Repository: github.com/Prinss-git/FaceLock

1. Getting Started & Git Branch Setup

All updates for the face recognition server, hardware firmware, and locker link architecture have been committed and pushed to the **hardware-ready** branch. When pulling this repository, team members must switch to this branch:

```
git fetch origin
git checkout hardware-ready
git pull origin hardware-ready
```

2. Credentials & Config Files (Not Stored in Git)

For security, credential files and API secrets are protected by .gitignore. Team members need to copy the corresponding templates and receive real credentials privately:

Target File	Source / Template	Contents / Notes
app/google-services.json	Firebase Console	Android configuration file generated in Firebase Project Settings.
recognizer/serviceAccountKey.json	Firebase Console	Service account private key (Project settings → Service accounts → Generate new private key).
recognizer/.env	recognizer/.env.example	Contains FIREBASE_PROJECT_ID, STORAGE_BUCKET_NAME, and shared DEVICE_KEY.
firmware/simulator/.env	firmware/simulator/.env.example	Web API key, Project ID, board user credentials (board-m-001@...), and locker ID M-001.
firmware/FaceLock_ESP32CAM/secrets.h	secrets.example.h	Wi-Fi credentials, Firebase Web API key, board auth, locker ID, and matching SECRET_RECOGNIZE_KEY.

3. Subsystem Instructions

A. Firebase Setup & Board User

- **Deploy Rules:** Run `firebase deploy --only firestore:rules,database` inside the `firebase/` folder.
- **Create Board Auth:** In Firebase Authentication → Users → Add user: `board-m-001@facelock.device` with a secure password. Copy the resulting User UID.
- **Link Board to Locker:** In Firestore, add doc `devices/<UID>` with fields: `lockerId = "M-001"`, `label = "Demo board"`.

B. Android Mobile App

- Place `google-services.json` into the `app/` directory.
- Open the project in Android Studio, let Gradle sync, then run **Build** → **Build APK(s)** and install on test devices.
- Register 1 Admin account and 2 Member accounts (e.g. Alice & Bob). Create building & locker **M-001** in the app.
- Enroll both members' face photos through the member face enrollment screen in the app.

C. Face Recognizer Server (Python)

- Set up Python venv and install dependencies: `cd recognizer && python -m venv .venv && pip install -r requirements.txt`
- Download ONNX face models (YuNet + SFace): `python download_models.py`

- Start server: python server.py (monitors Firestore, converts enrolled user photos into 128-D face embeddings, and serves unlock requests).
- Verify with webcam: python webcam_test.py --uid <uid> to verify MATCH vs NO MATCH.

D. Firmware Simulator (Node.js)

- Test complete locker workflow before physical hardware arrives: cd firmware/simulator && npm install && node simulate.js
- Verifies heartbeat, real-time unlock commands, offline indicators, and event logging in the mobile app.

4. Hardware Parts Checklist & Wiring

Refer to **firmware/WIRING.md** for complete schematic diagrams. Essential hardware bill of materials:

Component	Specification / Purpose	Pin / Wiring Connection
ESP32-CAM	AI Thinker module (OV2640 camera)	Main controller (5V, GND)
USB Adapter	ESP32-CAM-MB micro-USB daughterboard	For flashing & serial monitoring
Relay Module	5V 1-Channel Relay (active LOW)	IN to GPIO 12 (or GPIO 2 fallback)
Solenoid Lock	12V DC solenoid latch mechanism	Controlled via Relay COM & NO contacts
Flyback Diode	1N4007 diode across solenoid leads	Band/Cathode to +12V, Anode to GND
LCD Display	1602 LCD with I2C backpack (0x27)	SDA → GPIO 13, SCL → GPIO 14
PIR Sensor	HC-SR501 motion sensor	OUT → GPIO 15 (power from 5V)
Power Supply	12V 2A wall adapter + LM2596 step-down	Dial LM2596 to strictly 5.0V before connecting ESP32

5. Next Plans & Action Distribution

Role / Member	Primary Focus	Deliverables / Immediate Next Steps
Backend / Firebase Lead	Cloud rules, accounts & keys	1. Deploy firestore rules & test Spark plan quotas. 2. Create board auth account & link document in devices/. 3. Securely provide google-services.json & keys to team.
Mobile App Lead	Android App & enrollment UX	1. Build APK and distribute to team. 2. Create Admin and 2 Member accounts. 3. Test face enrollment and photo upload on mobile.
AI / ML Lead	Recognition engine & verification	1. Run download_models.py and verify local ONNX files. 2. Run python server.py to verify photo-to-embedding sync. 3. Test matching using webcam_test.py.
Hardware Lead	ESP32-CAM assembly & flashing	1. Set LM2596 step-down output to 5.0V with multimeter. 2. Assemble circuit per WIRING.md (tie all grounds together). 3. Flash FaceLock_ESP32CAM.ino and verify serial heartbeat.

6. Live Demonstration Day Checklist

Follow this sequential checklist for a seamless project presentation:

- **Hotspot Setup:** Connect laptop, ESP32-CAM board, and testing phones to the same 2.4 GHz phone hotspot.
- **Server Startup:** Run python server.py on laptop and verify it displays *"Listening for recognition requests"*.
- **Firewall Rule:** Verify Windows Firewall allows incoming Python connections across the hotspot network.
- **Locker Online Check:** Power on ESP32-CAM and check the Android app to ensure locker **M-001** status turns green (Online).
- **Remote App Unlock:** Tap Remote Unlock in the app; confirm relay clicks, solenoid pulls in, and audit log records the unlock.

- **Authorized Face Test:** Stand in front of camera; sensor triggers, LCD displays *"Recognizing..."*, locker unlocks, LCD shows member name.
- **Unauthorized Face Test:** Show an unregistered face; LCD displays *"Access Denied"*, red indicator/buzzer alerts, attempt logged.
- **Offline Detection Test:** Disconnect board power; confirm mobile app indicates locker *"Offline"* within 2 minutes.